

Rapport de stage

Détection et estimation 3D d'une balle rapide par caméra événementielle

Rigon Ajvazi

Mai 2026

Table des matières

1	Introduction	3
1.1	Contexte du stage	3
1.2	Objectifs techniques	4
2	Architecture générale du projet	5
2.1	Chaîne de traitement	5
2.2	Calibration et repère 3D	5
2.3	Visualisation	6
3	Première méthode : détection par circle fitting	7
3.1	Principe	7
3.2	Trajectoire 3D	8
3.3	Limite principale : sensibilité du rayon	9
4	Seconde méthode : estimation par trace événementielle	11
4.1	Motivation	11
4.2	Pipeline de la méthode Trace	12
4.3	Détection des bords soutenus	12
4.4	Conversion 3D par largeur de trace	12
4.5	Régression pondérée de trajectoire	13
5	Simulation et validation	14
5.1	Séquences simulées	14
5.2	Rôle des tests	14
6	Partie robotique : Isaac Lab et PPO	16
6.1	Objectif de la partie robot	16

6.2	Résultats d'entraînement observés	17
6.3	Défis pour l'intégration perception-robot	17
7	Bilan provisoire	18

Chapitre 1

Introduction

1.1 Contexte du stage

Le projet de stage s'inscrit dans le domaine de la robotique intelligente et de la perception en temps réel. L'objectif général est de permettre à un robot manipulateur d'attraper une balle lancée vers lui. Pour réaliser cette tâche, le robot doit disposer d'une estimation rapide et fiable de la position de la balle, puis utiliser cette information pour choisir une action d'interception.

Le problème est fortement dynamique. Une balle lancée se déplace rapidement, sa position change en quelques millisecondes, et une erreur de profondeur peut déplacer le point d'interception de plusieurs centimètres. La perception doit donc fonctionner avec une latence faible, tout en restant robuste au bruit, aux erreurs de calibration et aux déformations produites par le mouvement rapide de la balle.

Estimer la position de la balle

Utilisation d'une camera event pour l'estimation



FIGURE 1.1 – Caméra événementielle utilisée pour l'estimation de la position de la balle.

La solution étudiée repose sur une caméra événementielle DVXplorer. Contrairement à une caméra classique, elle ne produit pas une image complète à fréquence fixe. Elle émet des événements lorsqu'un changement local de luminosité est détecté. Chaque événement contient une position dans l'image, un timestamp et une polarité. Cette représentation est très adaptée aux objets rapides, car elle fournit une information temporelle fine et ne transmet que les pixels qui changent.

1.2 Objectifs techniques

Les objectifs du projet peuvent être résumés en quatre axes :

- détecter la balle dans un flux d'événements bruité ;
- estimer sa position 3D à partir de la calibration caméra et du rayon connu de la balle ;
- reconstruire une trajectoire cohérente dans le temps ;
- préparer l'utilisation de cette trajectoire par un robot entraîné dans Isaac Lab avec PPO.

La première approche développée repose sur un *circle fitting* : on ajuste un cercle sur les événements associés à la balle, puis on utilise le rayon apparent pour calculer la profondeur. Cette méthode fonctionne lorsque la projection de la balle reste proche d'un cercle. Cependant, pour une balle rapide, les événements forment souvent une trace allongée. Le rayon estimé devient alors instable et la profondeur calculée peut être fortement faussée.

Une seconde méthode a donc été développée : l'analyse de trace. Au lieu de chercher uniquement un cercle, elle exploite la trace complète laissée par la balle dans les événements. La largeur de cette trace est mesurée à partir de nombreux points, ce qui rend l'estimation moins dépendante d'une erreur locale de quelques pixels.

Chapitre 2

Architecture générale du projet

2.1 Chaîne de traitement

Le projet principal est un package C++/ROS 2. Il peut fonctionner avec la caméra DVXplorer réelle ou avec des fichiers d'événements enregistrés au format H5. La chaîne de traitement utilisée pendant le stage est la suivante :

1. acquisition des événements depuis la caméra ou lecture depuis un fichier ;
2. filtrage du bruit de fond ;
3. correction géométrique des points par la calibration OpenCV ;
4. conservation d'une fenêtre temporelle récente ;
5. échantillonnage pour limiter la charge de calcul ;
6. regroupement spatial des événements en clusters ;
7. estimation de la balle par cercle ou par trace ;
8. reconstruction d'une trajectoire 3D ;
9. visualisation et publication ROS 2 de la position estimée.

Le code évite de traiter directement toute l'image comme une image dense. Il conserve la logique événementielle : seuls les événements utiles sont filtrés, regroupés puis analysés. Cela permet de garder un temps de calcul faible, compatible avec une boucle de traitement très rapide.

2.2 Calibration et repère 3D

La calibration de la caméra réelle est chargée depuis un fichier OpenCV. Elle contient notamment les paramètres intrinsèques :

$$f_x \approx 761 \text{ px}, \quad f_y \approx 754 \text{ px}, \quad c_x \approx 305 \text{ px}, \quad c_y \approx 196 \text{ px}.$$

Les séquences simulées utilisent une calibration différente, fournie dans les fichiers de séquence :

$$f_x = f_y = 520 \text{ px}, \quad c_x = 320 \text{ px}, \quad c_y = 240 \text{ px}.$$

Cette distinction est importante : la calibration physique de la DVXplorer ne doit pas être utilisée pour les séquences simulées. Le lecteur de séquences charge automatiquement l'intrinsèque associé au fichier d'événements.

2.3 Visualisation

L'application fournit plusieurs vues : une vue 2D, une vue 3D, une vue de dessus, une vue de qualité RMSE et une vue Trace. Ces vues servent surtout au diagnostic. Elles permettent de vérifier si les clusters sont cohérents, si le cercle ajusté suit la balle, si la trajectoire 3D est plausible et si la trace événementielle est correctement exploitée.

Chapitre 3

Première méthode : détection par circle fitting

3.1 Principe

La première méthode considère que la balle est une sphère dont la projection dans l'image peut être approchée par un cercle. Après filtrage et regroupement des événements, le tracker sélectionne un groupe de points correspondant probablement à la balle. Un ajustement de cercle est ensuite effectué sur ce nuage de points.

Le cercle estimé fournit trois informations principales :

$$(u, v, r),$$

où u et v sont les coordonnées du centre dans l'image et r est le rayon apparent en pixels. En connaissant le rayon réel R de la balle et la focale f_x , la profondeur est estimée par :

$$Z = \frac{f_x R}{r}.$$

La position 3D complète est ensuite calculée par projection inverse :

$$X = \frac{u - c_x}{f_x} Z, \quad Y = \frac{v - c_y}{f_y} Z.$$

Dans le projet, la balle utilisée dans les séquences simulées possède un rayon de 20 mm. Le même ordre de grandeur est utilisé pour l'estimation réelle.

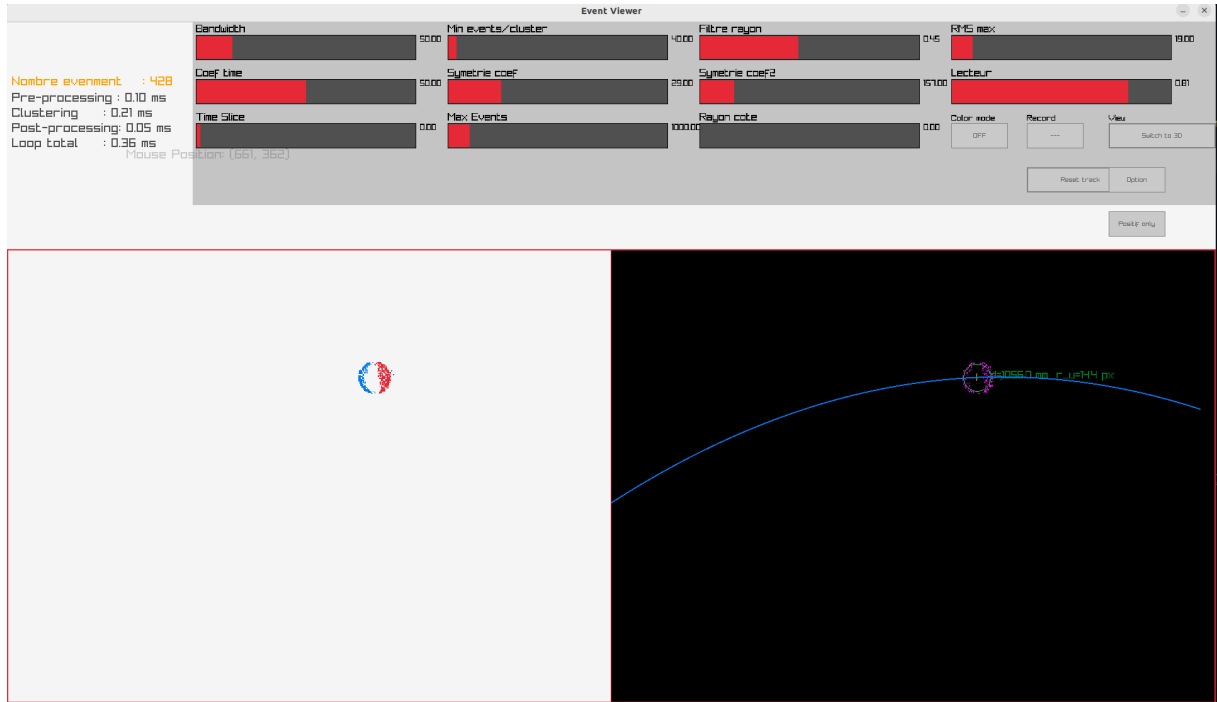


FIGURE 3.1 – Détection 2D par circle fitting : événements, cercle ajusté et trajectoire image.

3.2 Trajectoire 3D

Chaque position 3D valide est ajoutée à l'historique. La trajectoire est ensuite modélisée par des régressions simples :

$$x(t) = a_x t + b_x, \quad y(t) = a_y t + b_y, \quad z(t) = a_z t^2 + b_z t + c_z.$$

Ce choix correspond au mouvement attendu d'un projectile : les coordonnées latérales évoluent de manière approximativement linéaire, tandis que l'axe vertical suit une évolution parabolique sous l'effet de la gravité. L'objectif n'est pas seulement d'afficher les points mesurés, mais aussi de prédire la suite de la trajectoire pour préparer l'interception robotique.

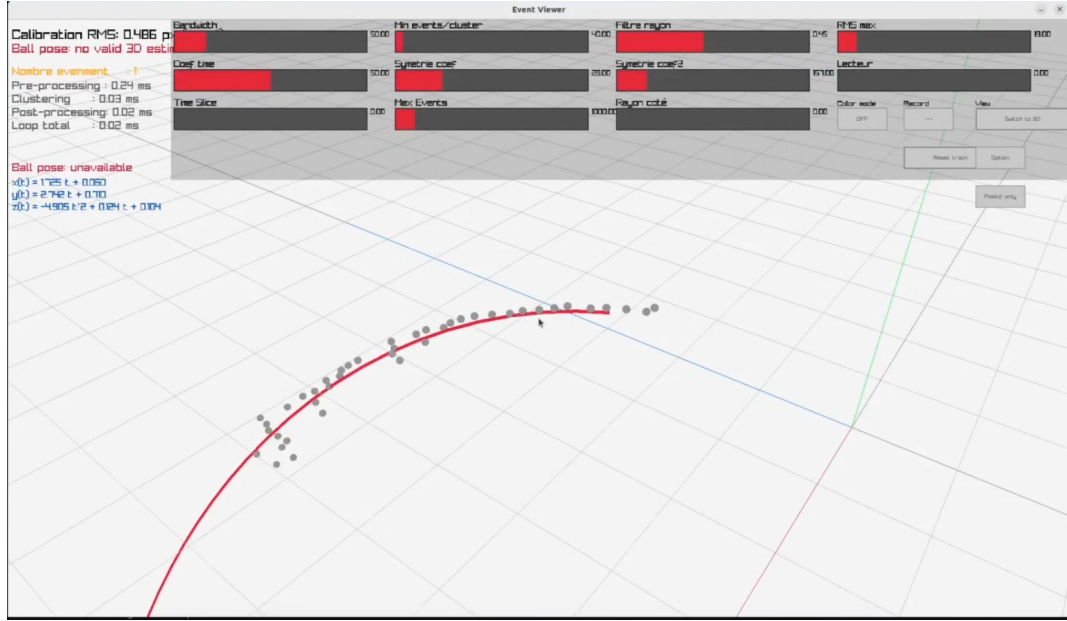


FIGURE 3.2 – Reconstruction de trajectoire 3D à partir des positions estimées par circle fitting.

3.3 Limite principale : sensibilité du rayon

La faiblesse de cette méthode vient du fait que toute la profondeur dépend du rayon apparent r . Une petite erreur sur r produit une grande erreur sur Z , surtout lorsque la balle est éloignée et que son rayon en pixels est faible.

À partir de :

$$Z = \frac{f_x R}{r},$$

on obtient la sensibilité :

$$\frac{dZ}{dr} = -\frac{f_x R}{r^2} = -\frac{Z}{r}.$$

Cela signifie qu'une erreur de 1 px sur le rayon produit environ Z/r d'erreur sur la profondeur.

TABLE 3.1 – Sensibilité de la profondeur pour la caméra réelle, avec $f_x \approx 761$ px et $R = 20$ mm.

Z	r image	Largeur	Erreur 1 px rayon	Erreur 1 px largeur
1.00 m	15.23 px	30.45 px	6.6 cm	3.3 cm
1.47 m	10.36 px	20.71 px	14.2 cm	7.1 cm
2.00 m	7.61 px	15.23 px	26.3 cm	13.1 cm

À 1.47 m, le rayon apparent vaut seulement environ 10.36 px. Une erreur d'un pixel sur ce rayon représente donc environ 14 cm d'erreur de profondeur. Si l'on raisonne sur une largeur ou un diamètre, l'erreur d'un pixel correspond à environ 7 cm. Cette sensibilité explique pourquoi le circle fitting devient insuffisant pour une balle rapide.

En pratique, la caméra événementielle ne voit pas toujours un contour circulaire net. Lors d'un mouvement rapide, les événements s'étalent dans le temps et forment une traînée. Le cercle ajusté peut alors surestimer le rayon apparent. Comme Z est inversement proportionnel à r , un rayon surestimé donne une balle artificiellement trop proche.

Chapitre 4

Seconde méthode : estimation par trace événementielle

4.1 Motivation

La méthode par trace part d'une observation simple : pour une balle rapide, l'information utile n'est pas seulement contenue dans un cercle instantané. Elle est répartie dans toute la trace événementielle produite pendant le déplacement de la balle. Cette trace contient beaucoup plus de points qu'un seul contour local.

L'idée est donc de mesurer la largeur réelle de cette trace, puis d'utiliser cette largeur comme diamètre apparent de la balle. Cette approche est plus robuste car elle s'appuie sur de nombreux événements et sur une cohérence spatiale globale.

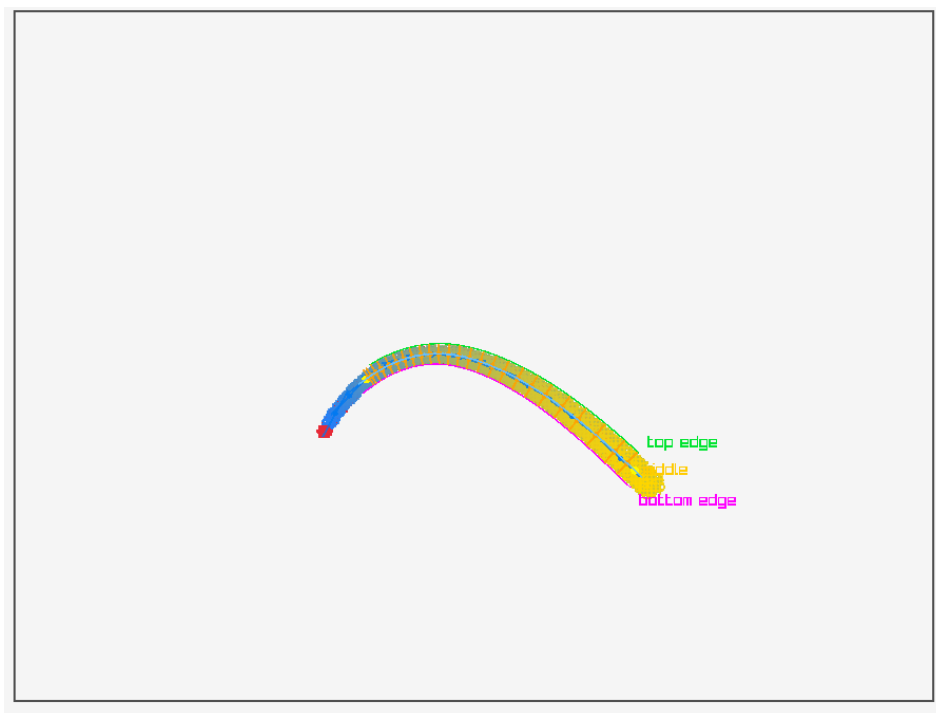


FIGURE 4.1 – Vue Trace : bord haut, ligne médiane, bord bas et mesures de largeur perpendiculaires.

4.2 Pipeline de la méthode Trace

La méthode Trace suit les étapes suivantes :

1. une fenêtre mobile suit la position courante de la balle ;
2. les événements récents qui tombent dans cette fenêtre sont accumulés ;
3. une PCA globale estime la direction principale de la trace ;
4. des PCA temporelles locales améliorent l'orientation lorsque la trace est courbe ;
5. les points sont projetés dans un repère local (s, h) ;
6. la trace est découpée en tranches selon s ;
7. dans chaque tranche, les bords sont détectés sur l'axe normal h ;
8. trois courbes sont ajustées : bord haut, ligne médiane et bord bas ;
9. la largeur est mesurée perpendiculairement à la ligne médiane ;
10. cette largeur est convertie en position 3D.

Le repère local est défini par :

$$s = (p - c) \cdot d, \quad h = (p - c) \cdot n,$$

où p est un événement image, c est le centre robuste de la trace, d la direction principale et n la normale. Cette transformation convertit un problème 2D difficile en une série de problèmes 1D : pour chaque tranche le long de s , il faut trouver les deux bords sur l'axe h .

4.3 Détection des bords soutenus

La détection des bords ne prend pas simplement le minimum et le maximum des valeurs h . Ces extrêmes peuvent être des points isolés. La méthode cherche plutôt les premiers bords qui possèdent assez de voisins proches dans un petit rayon local. Un bord est donc accepté seulement s'il est soutenu par plusieurs événements.

Cette logique répond mieux au bruit événementiel. Un point isolé à l'extérieur de la trace ne suffit pas à élargir artificiellement la mesure. La largeur locale est ensuite :

$$\text{largeur} = h_{\text{haut}} - h_{\text{bas}}.$$

Les largeurs locales aberrantes sont rejetées, puis les positions 3D obtenues sont utilisées pour reconstruire une trajectoire.

4.4 Conversion 3D par largeur de trace

La conversion 3D repose sur le même principe géométrique que le circle fitting, mais la grandeur mesurée n'est plus le rayon. La largeur de trace est utilisée comme diamètre apparent :

$$Z = \frac{f_{\text{eff}} D}{\text{largeur}_{px}},$$

où D est le diamètre réel de la balle et f_{eff} est la focale effective dans la direction normale utilisée pour mesurer la largeur. Le centre de la mesure permet ensuite de retrouver les coordonnées X et Y .

L'avantage est que la largeur est estimée à partir de nombreuses coupes de la trace. L'erreur d'un événement isolé influence beaucoup moins le résultat que dans le cas d'un cercle ajusté sur une forme déformée par la vitesse.

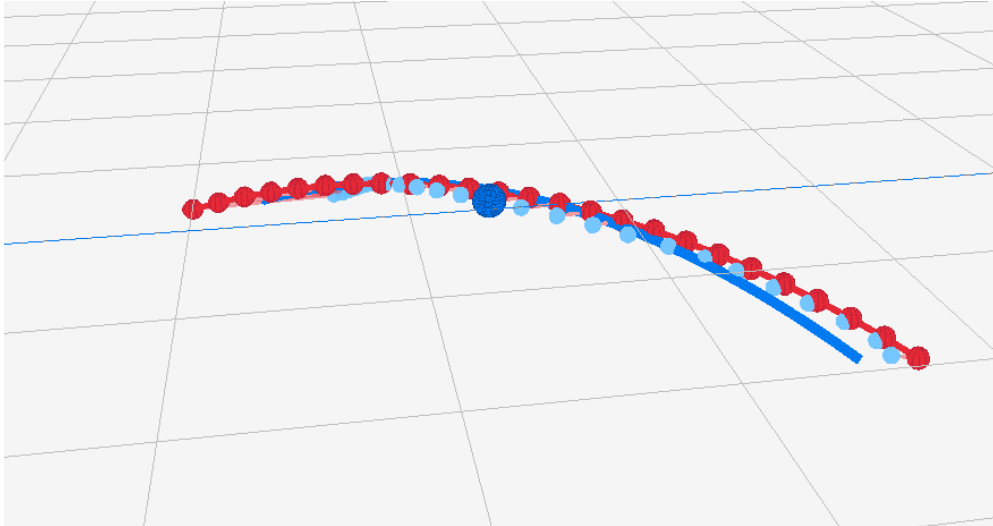


FIGURE 4.2 – Résultat 3D obtenu avec la méthode Trace. Les points estimés sont comparés à la trajectoire de référence lorsque celle-ci est disponible.

4.5 Régression pondérée de trajectoire

Une régression pondérée peut être activée pour la trajectoire finale. Le modèle géométrique reste le même :

$$x(t) = at + b, \quad y(t) = at + b, \quad z(t) = at^2 + bt + c.$$

La différence vient du poids associé à chaque observation. Les points récents reçoivent plus de poids, ce qui rend la trajectoire plus réactive. Les points dont le résidu est élevé reçoivent moins de poids, ce qui limite l'influence des mesures aberrantes. Cette pondération ne remplace pas la détection de trace, mais elle stabilise la trajectoire finale lorsque quelques poses 3D sont moins fiables.

Chapitre 5

Simulation et validation

5.1 Séquences simulées

Pour tester l'algorithme dans un cadre contrôlé, des séquences simulées ont été générées. La balle est simulée dans un environnement 3D, puis la vidéo MP4 est convertie en événements avec **v2e**. Cette méthode permet d'obtenir à la fois un flux événementiel et une vérité terrain de la trajectoire.

Les séquences locales utilisées dans le projet ont les caractéristiques suivantes :

- résolution vidéo : 640×480 ;
- fréquence d'entrée : 500 fps ;
- rayon de balle : 0.02 m ;
- calibration synthétique : $f_x = f_y = 520$ px ;
- résolution temporelle des événements : $200 \mu s$;
- seuils de génération ON/OFF : 0.15.

TABLE 5.1 – Séquences simulées disponibles localement.

Séquence	Durée	Événements bruts	Événements filtrés
sequence_0001	2.0 s	49 232	38 981
sequence_0002	2.0 s	47 835	37 909
sequence_0003	0.468 s	28 016	22 478

La vérité terrain contient la position réelle de la balle dans le repère caméra, ainsi que sa projection image. Elle permet de comparer la position estimée par l'algorithme avec la trajectoire attendue. Cette comparaison est importante, car une trajectoire peut sembler visuellement cohérente tout en ayant une erreur de profondeur trop élevée pour l'interception robotique.

5.2 Rôle des tests

Les séquences simulées servent à séparer deux problèmes :

- la qualité de l'algorithme géométrique ;
- les difficultés liées au matériel réel, à l'éclairage et au bruit de la caméra.

Lorsque l'algorithme échoue sur une séquence simulée avec vérité terrain, l'erreur vient principalement du traitement ou du modèle géométrique. Lorsqu'il fonctionne en simulation mais devient instable sur la caméra réelle, il faut plutôt analyser la calibration, la polarité des événements, la densité de la trace et les conditions expérimentales.

Chapitre 6

Partie robotique : Isaac Lab et PPO

6.1 Objectif de la partie robot

La perception seule ne suffit pas pour attraper la balle. Elle doit être intégrée à une commande robotique capable de déplacer l'effecteur vers une position d'interception. Cette partie a été étudiée dans Isaac Lab avec un apprentissage par renforcement de type PPO, pour *Proximal Policy Optimization*.

L'objectif de PPO est d'apprendre une politique de commande. À partir de l'état courant du robot et de la balle, la politique choisit une action qui rapproche le robot d'une situation favorable pour intercepter la balle. La simulation permet de répéter un grand nombre d'essais sans risque pour le matériel réel.

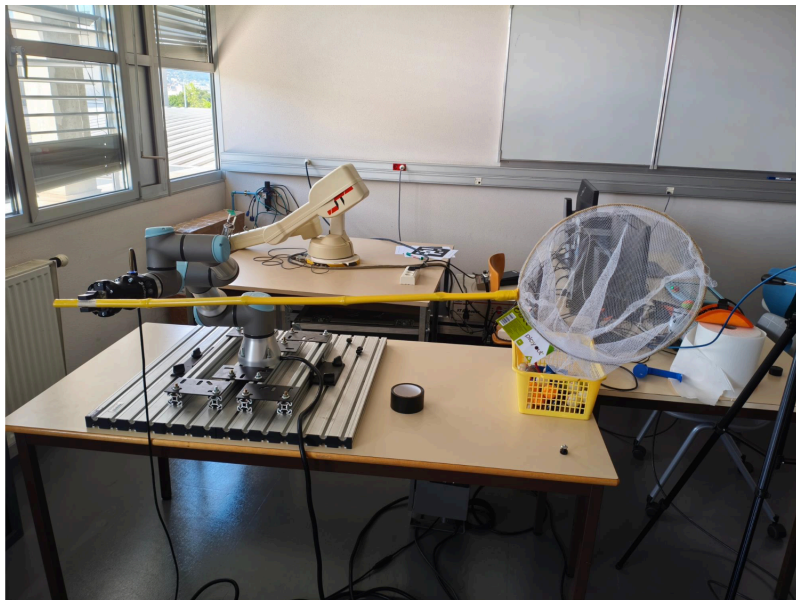


FIGURE 6.1 – Montage réel avec robot, caméra et zone de réception.

6.2 Résultats d'entraînement observés

D'après les essais présentés pendant le stage, l'entraînement atteint une convergence autour de 15 000 itérations. L'entraînement complet a été poussé jusqu'à environ 35 000 itérations. Avec 32 rollouts et environ 12 000 environnements, cela représente approximativement :

$$15\,000 \times 32 \times 12\,000 = 5.76 \times 10^9$$

steps de simulation pour la phase de convergence. Le temps total observé est d'environ 26 minutes, avec environ 11 minutes pour atteindre la convergence.

Ces chiffres montrent que la simulation permet d'entraîner rapidement un comportement de base. Cependant, le passage vers le système réel demande de prendre en compte plusieurs défis supplémentaires.

6.3 Défis pour l'intégration perception-robot

TABLE 6.1 – Principaux défis pour passer de la perception à l'interception robotique.

Défi	Impact attendu
Latence perception-commande	Une estimation correcte mais trop tardive peut déplacer le point d'interception. Il faut mesurer le temps entre acquisition, traitement, publication ROS 2 et action robot.
Synchronisation temporelle	La trajectoire doit être associée à des timestamps fiables. Sans synchronisation, le robot peut suivre une balle déjà passée.
Calibration caméra-robot	La position 3D estimée dans le repère caméra doit être transformée dans le repère du robot avec une extrinsèque précise.
Espace atteignable	Toutes les trajectoires de balle ne sont pas interceptables. Il faut vérifier la zone de travail, les limites articulaires et la vitesse maximale du robot.
Bruit de mesure	Une erreur de quelques centimètres en profondeur peut suffire à rater la balle. La méthode Trace réduit ce risque, mais un filtrage temporel reste nécessaire.
Sim-to-real	Une politique apprise en simulation peut échouer si la dynamique réelle, les délais ou les observations diffèrent trop du simulateur.
Sécurité	Les essais réels doivent limiter les vitesses, gérer les arrêts d'urgence et éviter les trajectoires dangereuses du robot.

La suite logique du projet consiste donc à connecter la perception Trace à la partie robotique. La position de balle estimée doit être convertie dans le repère du robot, filtrée, prédite, puis utilisée comme entrée de la politique de commande ou d'un contrôleur d'interception.

Chapitre 7

Bilan provisoire

Le travail réalisé a permis de construire une chaîne complète de perception événementielle pour une balle rapide. La première méthode, fondée sur le circle fitting, donne une base simple et efficace lorsque la balle produit un contour circulaire propre. Elle montre toutefois une limite forte : la profondeur dépend directement du rayon apparent, et une erreur d'un pixel peut représenter plusieurs centimètres, voire davantage lorsque la balle est loin.

La méthode Trace répond à cette difficulté en exploitant la forme réellement produite par une balle rapide dans une caméra événementielle. Au lieu de forcer un cercle sur une trace allongée, elle mesure une largeur cohérente à partir de nombreux événements. Cette approche est plus logique pour le signal disponible et réduit la sensibilité aux points isolés.

Les séquences simulées issues de vidéos MP4 converties en événements permettent de tester l'algorithme avec une vérité terrain. Elles constituent une étape importante avant l'intégration complète avec le robot. Enfin, les essais PPO dans Isaac Lab montrent que la partie robotique peut apprendre un comportement d'interception, mais l'intégration réelle devra traiter avec soin la latence, la calibration et la robustesse de la trajectoire.